

Veiledning for digitale sikkerhetsberegninger med modellraketter i skolen

Introduksjon: Denne veiledningen er utviklet for å støtte lærere i planlegging og gjennomføring av praktisk og utforskende naturfagsundervisning med modellraketter. Slike prosjekter gir unike muligheter for å arbeide med utforskende og elevaktiv naturfagsundervisning, og samsvarer godt med fagets kjerneelementer, sentrale verdier og mange konkrete læreplanmål i LK20. Behovet for robuste sikkerhetsvurderinger og dokumentasjon av disse kan utgjøre en betydelig barriere for å ta i bruk modellraketter i skolehverdagen.

Dette verktøyet er utformet som et pedagogisk stilas som veileder brukeren steg for steg gjennom de nødvendige fasene av sikkerhetsarbeidet. Innholdet er strukturert kronologisk fra innledende beregninger, vurdering av prosjektets gjennomførbarehet, mer detaljerte simuleringer i designfasen, til praktiske sjekklister og prosedyrer for selve oppskytingsdagen.

Veiledningen er et resultat av et designvitenskapelig utviklingsarbeid med lærere, og er utviklet som en del av et mastergradsprosjekt ved OsloMet - Storbyuniversitetet, med mål om å redusere barrierer, og trygge lærere i sitt arbeid med modellraketter.

Dette produktet er en prototype som vil bli videreutviklet fra høsten 2026. Oppdateringer vil bli publisert på https://github.com/MortenOsterud/modellraketter_i_skolen

Jeg har som mål at denne veiledningen kan både brukes direkte i undervisning med modellraketter i skolen, eller brukes som utgangspunkt for å utvikle andre veiledninger der digitale verktøy kan benyttes for sikkerhetsvurderinger. Ihht lisensen publisert på https://github.com/MortenOsterud/modellraketter_i_skolen/blob/main/LICENSE står du som bruker fritt til å videreutvikle eller bruke denne lærerveiledningen slik du selv måtte ønske i undervisning, forskning eller på andre måter, men vær oppmerksom på at enkelte vedlegg og programkode er gjengitt med tillatelse fra opphavspersonene, og hvis disse delene skal distribueres videre må opphavspersoner kontaktes direkte for godkjenning. Disse delene er tydelig merket.

Målgruppe for veiledningen: Lærere i skolen som ønsker å bruke modellraketter i undervisningen, men ønsker å se hvordan sikkerhetsberegninger kan gjøres for å sikre trygg oppskyting.

Det anbefales å laste ned alle vedleggene til denne veiledningen. Disse er fritt tilgjengelig for bruk for alle som ønsker å bruke dem i skolesammenheng. Ressurser fra Andøya Space Education er gjengitt med tillatelse.

Ved spørsmål til lærerveiledningen, ikke nøl med å ta kontakt på s141968@oslomet.no

Innhold

[Veiledning for digitale sikkerhetsberegninger med modellraketter i skolen](#)

[Beregninger i oppstartsfasen – Sannsynliggjøring av at prosjektet kan gjennomføres](#)

[Beregninger i designfasen](#)

[Andre beregninger](#)

[Beregninger før oppskyting](#)

[Manuelle sikkerhetsberegninger](#)

[Andre sikkerhetstiltak og sjekkliste](#)

[Før oppskytingsdagen](#)

[På oppskytingsdagen](#)

[Ekstra sikkerhetstiltak - Avfyringspanel](#)

[Tips til lærer:](#)

[Utstyr:](#)

[Programmering av sender](#)

[Programmering av mottaker](#)

[Bygging av avfyringspanel](#)

[Testing av avfyringspanel](#)

[Evaluerings av sikkerhetstiltak og kalibrering av beregninger etter oppskyting](#)

[Vedlegg](#)

Beregninger i oppstartsfasen – Sannsynliggjøring av at prosjektet kan gjennomføres

Før man kan skyte opp raketter i skolen er det mange faser av sikkerhetsvurderinger som må gjennomføres. Hvor mye tid og ressurser disse beregningene tar varierer. Derfor er det alltid hensiktsmessig å gjøre en første vurdering av hvorvidt prosjektet kan være gjennomførbart. Enkle fysikkberegninger kan hjelpe deg med å avgjøre om en rakett vil forlate launchpad, og hvor høyt den maksimalt kan gå.

La oss ta et ganske vanlig skoleekspempel: En liten modellrakett med en C eller D-klasse-motor. Før vi legger ned mangfoldige timer i design, bygging, forberedende pedagogisk arbeid etc må vi forsikre oss om at prosjektet er gjennomførbart. Ifølge dronekommandoen til politiet i Oslo må man ha følgende kriterier på plass om man ikke er sertifisert dronепilot og skal skyte opp en modellrakett i undervisningssammenheng uten spesielle tillatelser.

Sikkerhetskriterier:

Rakettmotorklasse: D eller lavere (20Ns eller mindre)

Totalvekt: Under 250g

Maksimal flyhøyde: 120 meter

Annet: Sikkerhetssone, tillatelse fra grunneier, risikovurdering etc

I første fase av sikkerhetsberegninger bruker jeg de fire bevegelseslikningene fra fysikk1 ettersom de fleste realfagslærere er kjent med disse:

Bevegelseslikninger

$a = \text{akselerasjon} \left(-9,81 \text{ m/s}^2 \text{ på jorda} \right)$

$s = \text{strekning (meter)} \quad t = \text{tid (sekunder)} \quad v_0 = \text{start fart (m/s)}$

$v = \text{slutt fart (m/s)}$

$$I : \dots\dots\dots v = at + v_0$$

$$II : \dots\dots\dots s = \frac{1}{2}(v + v_0)t$$

$$III : \dots\dots\dots 2as = v^2 - v_0^2$$

$$IV : \dots\dots\dots s = v_0t + \frac{1}{2}at^2$$

Når vi skal gjøre enkle fysikkberegninger har vi naturligvis mulighet til å gjøre disse for hånd, men her kan digitale verktøy spare både tid, og gjøre utregningene mer oversiktlige for oss.

Dessuten kan de gjøre beregningene mer tilgjengelige for elever, da de kan følge utregningene stegvis og kvalitetssikre at koden faktisk regner ut det man tror. Her er to svært grove beregninger jeg gjorde med en masse på 0.249kg og de to motortypene jeg har tilgjengelige: C6 og D9.

Kjør koden nedenfor ved å plassere markøren i cellen og trykke shift+enter

```
In [2]: total_masse = 0.249      # kg
        motor = 'D9'           # Enten 'D9', 'C6' eller 'B4'

def bestem_kraft_og_tid(motor):
    """Setter korrekt motorkraft og brenntid basert på motortype"""
    if motor == 'D9':
        motor_kraft = 9        # N (kgms^-2)
        motor_tid = 2.1        # s
    elif motor == 'C6':
        motor_kraft = 6        # N (kgms^-2)
        motor_tid = 1.6        # s
    elif motor == 'B4':
        motor_kraft = 4        # N (kgms^-2)
        motor_tid = 1.3        # s
    else:
        motor_kraft = 0
        motor_tid = 0

    return motor_kraft, motor_tid

def finn_oppskytningsakselerasjon(masse, kraft, g = 9.81):
    """Beregner konstant akselerasjonen for rakett skutt rett opp gitt motorkraft"""
    akselerasjon = kraft/masse - g
    return akselerasjon

def finn_slutfart(a, t, v_0 = 0):
```

```

    """Bevegelseslikning 1. Finner slutfart gitt akselerasjon, tid og evt startfar
    v = a*t + v_0
    return v

def finn_strekning(v, t, v_0 = 0):
    """Bevegelseslikning 2. Finner strekning gitt startfart, slutfart (valgfri) og
    s = 0.5*(v+v_0)*t
    return s

def finn_strekning2(v, a = 9.81, v_0 = 0):
    """Bevegelseslikning 3, omskrevet med hensyn på s. Finn strekning til apogee et
    s = (v**2-v_0**2)/(2*a)
    return s

motor_kraft, motor_tid = bestem_kraft_og_tid(motor)
oppskytningsakselerasjon = finn_oppskytningsakselerasjon(total_masse, motor_kraft)
v_motorstopp = finn_slutfart(oppskytningsakselerasjon, motor_tid)
h_motorstopp = finn_strekning(v_motorstopp, motor_tid)
h_coasting = finn_strekning2(v_motorstopp)
h_total = h_motorstopp+h_coasting

print(f"Akselerasjon: {round(oppskytningsakselerasjon,3)} m/s^2")
print(f"Maksfart: {round(v_motorstopp,2)} m/s")
print(f"Høyde MECO: {round(h_motorstopp,1)} m")
print(f"Høyde coast: {round(h_coasting,1)} m")
print(f"Total høyde u/luftmotstand: {round(h_total,1)}m")

```

Akselerasjon: 26.335 m/s²
 Maksfart: 55.3 m/s
 Høyde MECO: 58.1 m
 Høyde coast: 155.9 m
 Total høyde u/luftmotstand: 213.9m

Figur 1 – Oppskyting D9-motor med masse på 0.249 kg

Som vi ser av figur 1 vil raketten ha nok energi til å nå 213.9m ved bruk av D9-motor.

Nå kan vi forsøke å endre motortype til 'C6' for å se hvor høyt raketten vil kunne gå. Legg merke til at all annen kode er lik, bortsett fra linje 2, der motortypen er endret. Kjør koden ved å markere cellen og trykke shift+enter

```

In [4]: total_masse = 0.249      # kg
        motor = 'C6'           # Enten 'D9', 'C6' eller 'B4'

def bestem_kraft_og_tid(motor):
    """Setter korrekt motorkraft og brenntid basert på motortype"""
    if motor == 'D9':
        motor_kraft = 9          # N (kgms^-2)
        motor_tid = 2.1          # s
    elif motor == 'C6':
        motor_kraft = 6          # N (kgms^-2)
        motor_tid = 1.6          # s
    elif motor == 'B4':
        motor_kraft = 4          # N (kgms^-2)
        motor_tid = 1.3          # s
    else:
        motor_kraft = 0
        motor_tid = 0

    return motor_kraft, motor_tid

def finn_oppskytningsakselerasjon(masse, kraft, g = 9.81):

```



```

"""Beregner konstant akselerasjonen for rakett skutt rett opp gitt motorkraft"""
akselerasjon = kraft/masse - g
return akselerasjon

def finn_sluttfart(a, t, v_0 = 0):
    """Bevegelseslikning 1. Finner sluttfart gitt akselerasjon, tid og evt startfar
    v = a*t + v_0
    return v

def finn_strekning(v, t, v_0 = 0):
    """Bevegelseslikning 2. Finner strekning gitt startfart, sluttfart (valgfri) og
    s = 0.5*(v+v_0)*t
    return s

def finn_strekning2(v, a = 9.81, v_0 = 0):
    """Bevegelseslikning 3, omskrevet med hensyn på s. Finn strekning til apogee et
    s = (v**2-v_0**2)/(2*a)
    return s

motor_kraft, motor_tid = bestem_kraft_og_tid(motor)
oppskytningsakselerasjon = finn_oppskytningsakselerasjon(total_masse, motor_kraft)
v_motorstopp = finn_sluttfart(oppskytningsakselerasjon, motor_tid)
h_motorstopp = finn_strekning(v_motorstopp, motor_tid)
h_coasting = finn_strekning2(v_motorstopp)
h_total = h_motorstopp+h_coasting

print(f"Akselerasjon: {round(oppskytningsakselerasjon,3)} m/s^2")
print(f"Maksfart: {round(v_motorstopp,2)} m/s")
print(f"Høyde MECO: {round(h_motorstopp,1)} m")
print(f"Høyde coast: {round(h_coasting,1)} m")
print(f"Total høyde u/luftmotstand: {round(h_total,1)}m")

```

```

Akselerasjon: 14.286 m/s^2
Maksfart: 22.86 m/s
Høyde MECO: 18.3 m
Høyde coast: 26.6 m
Total høyde u/luftmotstand: 44.9m

```

Figur 2 – Oppskytning C6-motor med masse total på 0.249kg

Figur 2 viser at raketten kun vil kunne gå 44.9m ved bruk av C6-motor. Ettersom det i praksis ikke finnes noen mellomting mellom disse motorene til salgs i Norge vet jeg allerede at jeg må enten:

1. Øke massen til raketten for å få den til å gå lavere slik at den holder seg under 120 meter. Dette må avklares med politiet for å sikre at jeg har tillatelse og sertifiseringer til å skyte rakett med over 249 gram totalvekt.
2. Søke til departementet for samfunnssikkerhet og beredskap (DSB) om å få skyte en rakett (D9, 249g) som antageligvis vil gå et sted mellom 120 og 250 meter totalhøyde.
3. Redusere massen og beholde C6-motor for å få raketten til å nærme seg en beregnet totalhøyde på 120 meter.

For å utforske både alternativ 1 og 3 kan vi bruke samme beregninger sammen med matplotlib til å vise hvordan variasjoner i masse påvirker forventet høyde. Her er en revidert kode som bruker matplotlib til å vise forventet høyde med ulike masser med en gitt motor. De ulike beregningene har også blitt samlet i én funksjon.

```

In [5]: import matplotlib.pyplot as plt
import numpy as np

minimal_masse = 0.300 # kg
maksimal_masse = 0.350 # kg
motor = 'D9' # Enten 'D9', 'C6' eller 'B4'

def bestem_kraft_og_tid(motor):
    """Setter korrekt motorkraft og brenntid basert på motortype"""
    if motor == 'D9':
        motor_kraft = 9 # N (kgms-2)
        motor_tid = 2.1 # s
    elif motor == 'C6':
        motor_kraft = 6 # N (kgms-2)
        motor_tid = 1.6 # s
    elif motor == 'B4':
        motor_kraft = 4 # N (kgms-2)
        motor_tid = 1.3 # s
    else:
        motor_kraft = 0
        motor_tid = 0

    return motor_kraft, motor_tid

def finn_oppskytningsakselerasjon(masse, kraft, g = 9.81):
    """Beregner konstant akselerasjonen for rakett skutt rett opp gitt motorkraft"""
    akselerasjon = kraft/masse - g
    return akselerasjon

def finn_sluttfart(a, t, v_0 = 0):
    """Bevegelseslikning 1. Finner sluttfart gitt akselerasjon, tid og evt startfart"""
    v = a*t + v_0
    return v

def finn_strekning(v, t, v_0 = 0):
    """Bevegelseslikning 2. Finner strekning gitt startfart, sluttfart (valgfri) og"""
    s = 0.5*(v+v_0)*t
    return s

def finn_strekning2(v, a = 9.81, v_0 = 0):
    """Bevegelseslikning 3, omskrevet med hensyn på s. Finn strekning til apogee et"""
    s = (v**2-v_0**2)/(2*a)
    return s

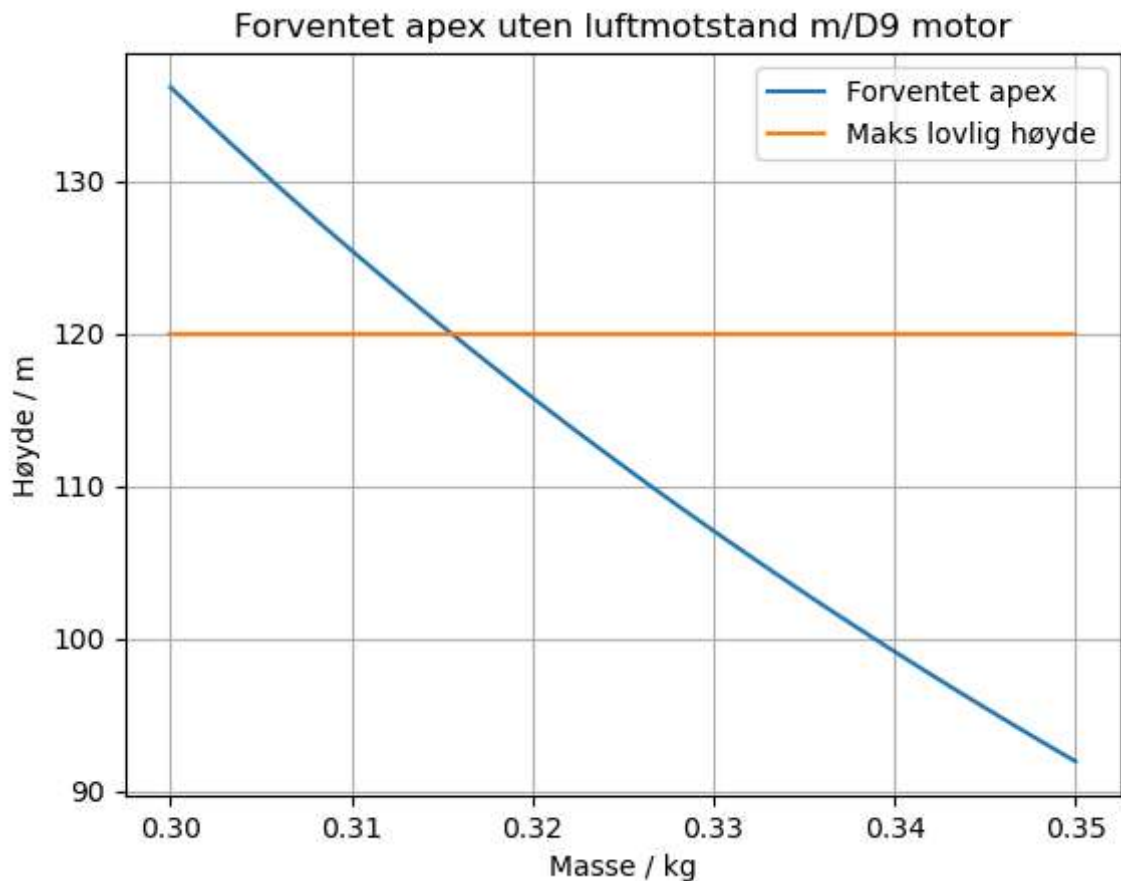
def finn_apex(total_masse, motor):
    """Finner apex ved oppskytning rett opp uten luftmotstand gitt masse og motor"""
    motor_kraft, motor_tid = bestem_kraft_og_tid(motor)
    oppskytningsakselerasjon = finn_oppskytningsakselerasjon(total_masse, motor_kraft)
    v_motorstopp = finn_sluttfart(oppskytningsakselerasjon, motor_tid)
    h_motorstopp = finn_strekning(v_motorstopp, motor_tid)
    h_coasting = finn_strekning2(v_motorstopp)
    h_total = h_motorstopp+h_coasting
    return h_total

masser = np.linspace(minimal_masse, maksimal_masse,100)
apex = finn_apex(masser,motor)
maks_lovlig = [120]*len(masser)

plt.plot(masser, apex, label = 'Forventet apex')
plt.plot(masser, maks_lovlig, label = 'Maks lovlig høyde')
plt.xlabel('Masse / kg')
plt.ylabel('Høyde / m')
plt.legend()

```

```
plt.grid()
plt.title(f'Forventet apex uten luftmotstand m/{motor} motor')
plt.show()
```



Figur 3 - Totalmasse mellom 300 og 350 gram m/D9-motor

Som vi kan se av figur 3 vil en økning på kun 50-100g fra totalmasse på 249g til totalmasse på 300-350g ha stor påvirkning på forventet oppskytingshøyde, og føre til at forventet høyde vil gå ned til det lovlige området. Ettersom de fleste lærere antageligvis ikke har dronepilotsertifisering, og derfor ikke kan skyte raketter over 249g ser jeg ikke nærmere på dette alternativet her. Nytteverdien av beregninger med andre masser og motorstyrker i pedagogisk sammenheng i klasserommet kan allikevel være stor.

```
In [6]: import matplotlib.pyplot as plt
import numpy as np

minimal_masse = 0.150 # kg
maksimal_masse = 0.250 # kg
motor = 'C6' # Enten 'D9', 'C6' eller 'B4'

def bestem_kraft_og_tid(motor):
    """Setter korrekt motorkraft og brenntid basert på motortype"""
    if motor == 'D9':
        motor_kraft = 9 # N (kgms^-2)
        motor_tid = 2.1 # s
    elif motor == 'C6':
        motor_kraft = 6 # N (kgms^-2)
        motor_tid = 1.6 # s
    elif motor == 'B4':
        motor_kraft = 4 # N (kgms^-2)
        motor_tid = 1.3 # s
    else:
```

```

        motor_kraft = 0
        motor_tid = 0

    return motor_kraft, motor_tid

def finn_oppskytningsakselerasjon(masse, kraft, g = 9.81):
    """Beregner konstant akselerasjonen for rakett skutt rett opp gitt motorkraft"""
    akselerasjon = kraft/masse - g
    return akselerasjon

def finn_sluttfart(a, t, v_0 = 0):
    """Bevegelseslikning 1. Finner sluttfart gitt akselerasjon, tid og evt startfar
     $v = a \cdot t + v_0$ 
    """
    v = a*t + v_0
    return v

def finn_strekning(v, t, v_0 = 0):
    """Bevegelseslikning 2. Finner strekning gitt startfart, sluttfart (valgfri) og
     $s = 0.5 \cdot (v + v_0) \cdot t$ 
    """
    s = 0.5*(v+v_0)*t
    return s

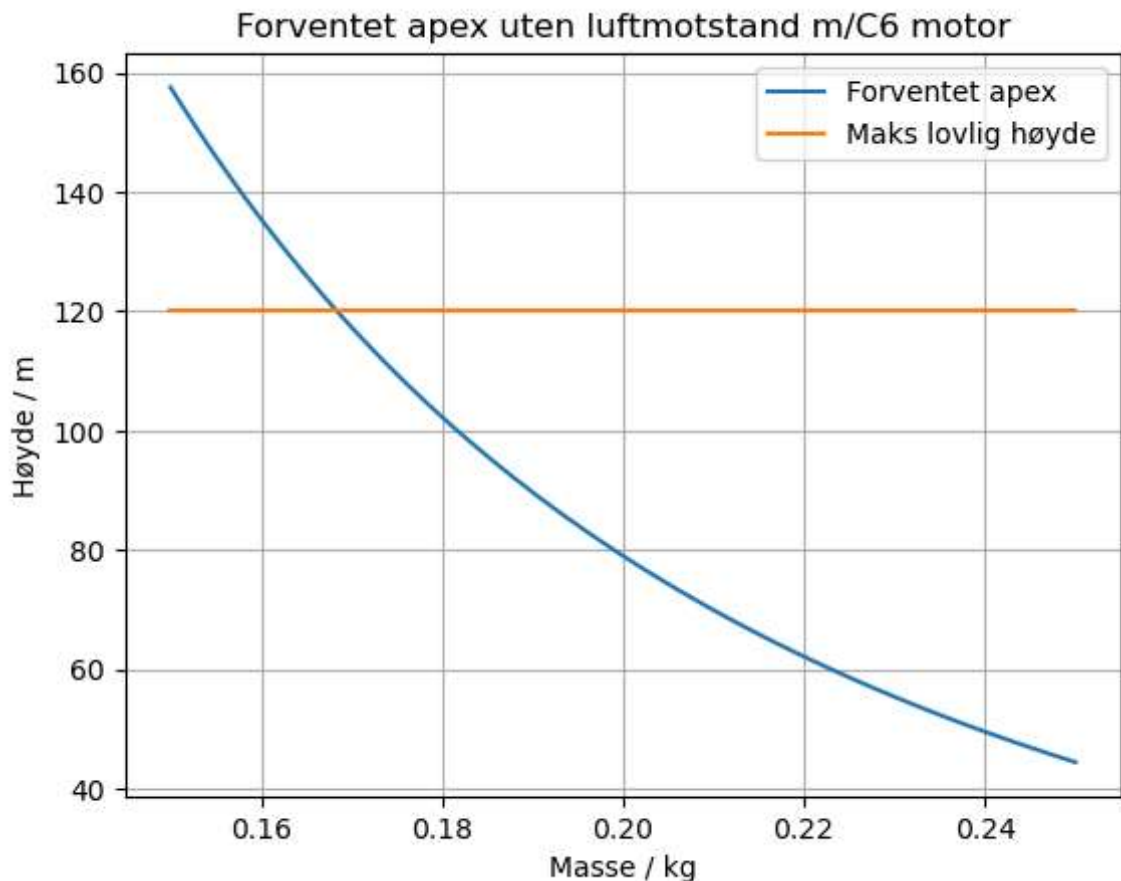
def finn_strekning2(v, a = 9.81, v_0 = 0):
    """Bevegelseslikning 3, omskrevet med hensyn på s. Finn strekning til apogee et
     $s = (v^2 - v_0^2) / (2 \cdot a)$ 
    """
    s = (v**2-v_0**2)/(2*a)
    return s

def finn_apex(total_masse, motor):
    """Finner apex ved oppskytning rett opp uten luftmotstand gitt masse og motor"""
    motor_kraft, motor_tid = bestem_kraft_og_tid(motor)
    oppskytningsakselerasjon = finn_oppskytningsakselerasjon(total_masse, motor_kraft)
    v_motorstopp = finn_sluttfart(oppskytningsakselerasjon, motor_tid)
    h_motorstopp = finn_strekning(v_motorstopp, motor_tid)
    h_coasting = finn_strekning2(v_motorstopp)
    h_total = h_motorstopp+h_coasting
    return h_total

masser = np.linspace(minimal_masse, maksimal_masse,100)
apex = finn_apex(masser,motor)
maks_lovlig = [120]*len(masser)

plt.plot(masser, apex, label = 'Forventet apex')
plt.plot(masser, maks_lovlig, label = 'Maks lovlig høyde')
plt.xlabel('Masse / kg')
plt.ylabel('Høyde / m')
plt.legend()
plt.grid()
plt.title(f'Forventet apex uten luftmotstand m/{motor} motor')
plt.show()

```



Figur 4 - Totalmasse mellom 150 og 250 gram m/C6-motor

Som vi kan se av figur 4 har en rakett med totalmasse på ca 170g med C6-motor ingen teoretisk mulighet til å gå over 120 meter, og dette er et godt utgangspunkt for videre sikkerhetsberegninger. Mer nøyaktige tall kommer vi frem til senere, men for referanse kan vi se av figur 5 nedenfor at dersom vi bruker en svakere B4-motor vil totalmassen måtte være rundt 95g, som er veldig lavt og begrenser oss med tanke på design og payload-muligheter.

```
In [7]: import matplotlib.pyplot as plt
import numpy as np

minimal_masse = 0.09 # kg
maksimal_masse = 0.11 # kg
motor = 'B4' # Enten 'D9', 'C6' eller 'B4'

def bestem_kraft_og_tid(motor):
    """Setter korrekt motorkraft og brenntid basert på motortype"""
    if motor == 'D9':
        motor_kraft = 9 # N (kgms^-2)
        motor_tid = 2.1 # s
    elif motor == 'C6':
        motor_kraft = 6 # N (kgms^-2)
        motor_tid = 1.6 # s
    elif motor == 'B4':
        motor_kraft = 4 # N (kgms^-2)
        motor_tid = 1.3 # s
    else:
        motor_kraft = 0
        motor_tid = 0

    return motor_kraft, motor_tid
```

```

def finn_oppskytningsakselerasjon(masse, kraft, g = 9.81):
    """Beregner konstant akselerasjonen for rakett skutt rett opp gitt motorkraft"""
    akselerasjon = kraft/masse - g
    return akselerasjon

def finn_sluttfart(a, t, v_0 = 0):
    """Bevegelseslikning 1. Finner sluttfart gitt akselerasjon, tid og evt startfart"""
    v = a*t + v_0
    return v

def finn_strekning(v, t, v_0 = 0):
    """Bevegelseslikning 2. Finner strekning gitt startfart, sluttfart (valgfri) og tid"""
    s = 0.5*(v+v_0)*t
    return s

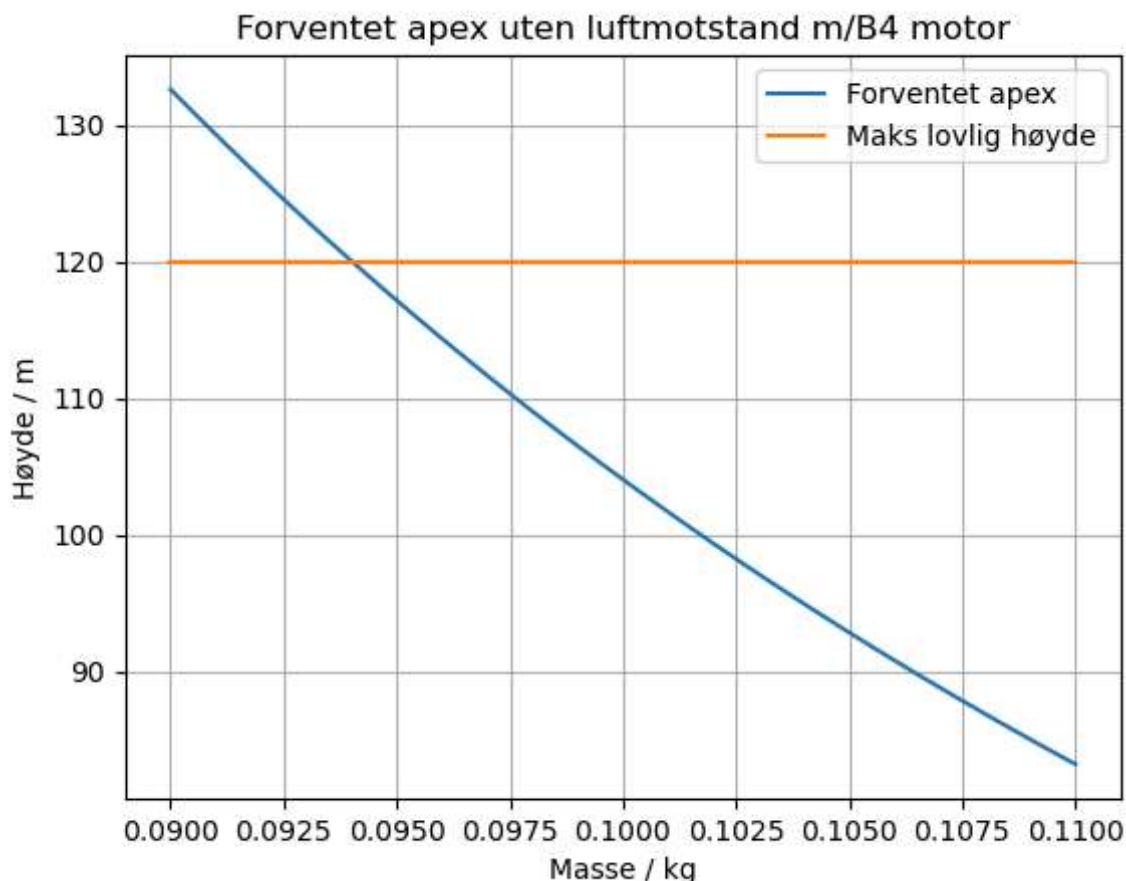
def finn_strekning2(v, a = 9.81, v_0 = 0):
    """Bevegelseslikning 3, omskrevet med hensyn på s. Finn strekning til apogee ettersom v=0"""
    s = (v**2-v_0**2)/(2*a)
    return s

def finn_apex(total_masse, motor):
    """Finner apex ved oppskytning rett opp uten luftmotstand gitt masse og motor"""
    motor_kraft, motor_tid = bestem_kraft_og_tid(motor)
    oppskytningsakselerasjon = finn_oppskytningsakselerasjon(total_masse, motor_kraft)
    v_motorstopp = finn_sluttfart(oppskytningsakselerasjon, motor_tid)
    h_motorstopp = finn_strekning(v_motorstopp, motor_tid)
    h_coasting = finn_strekning2(v_motorstopp)
    h_total = h_motorstopp+h_coasting
    return h_total

masser = np.linspace(minimal_masse, maksimal_masse,100)
apex = finn_apex(masser,motor)
maks_lovlig = [120]*len(masser)

plt.plot(masser, apex, label = 'Forventet apex')
plt.plot(masser, maks_lovlig, label = 'Maks lovlig høyde')
plt.xlabel('Masse / kg')
plt.ylabel('Høyde / m')
plt.legend()
plt.grid()
plt.title(f'Forventet apex uten luftmotstand m/{motor} motor')
plt.show()

```

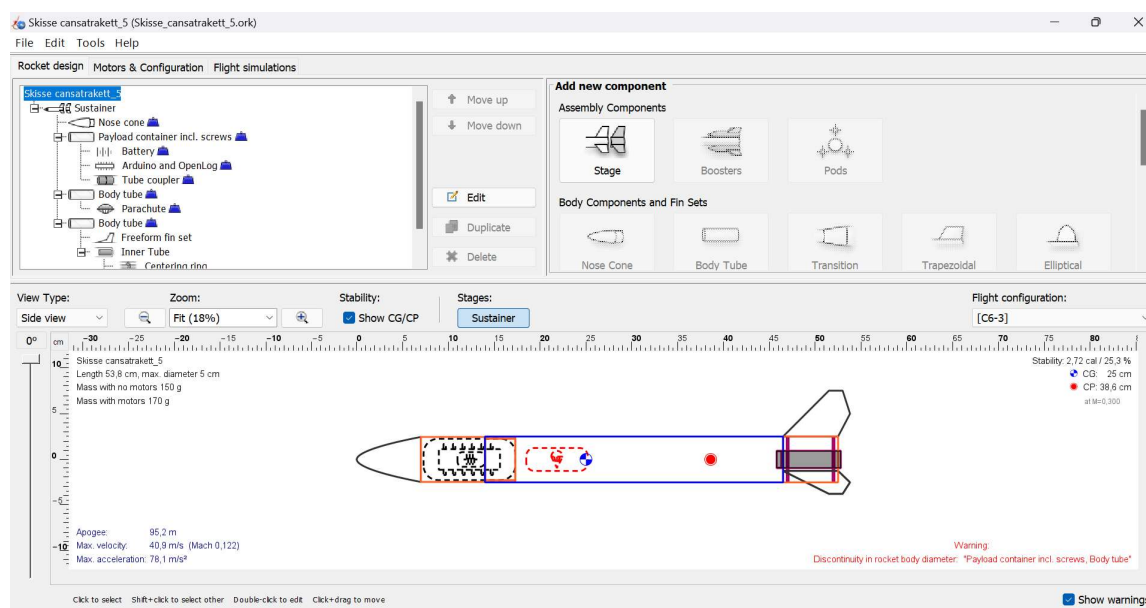
Figur 5 - Totalmasse 90-110g med B4-motor

På bakgrunn av disse analysene tar jeg i det videre arbeidet med sikkerhetsvurdering utgangspunkt i raketter rundt 170g med C6-motorer. Vi kan være ganske sikre på at vi kan holde oss innenfor trygge og lovlige rammer dersom vi designer raketten vår til å ha en totalvekt på ca 170g med en C6-motor på en vindstille dag. For å bli enda sikrere bør vi både dykke dypere ned i simuleringer og se på oppskytingsdata fra andre lignende oppskytninger.

Beregninger i designfasen

I designfasen av rakettprosjektet kan man velge å gå en relativt trygg vei ved å bruke ferdige rakettsett fra f.eks Klima, følge instruksjoner fra eksempelvis Andøya Space Centers strålende hefte *Modellraketter, Aktiviteter for fysikken i fokus* (vedlagt etter tillatelse fra Andøya Space Center) eller man kan designe raketter fra grunnen selv. Uavhengig av hvordan man går frem her er det helt nødvendig å gjøre sine egne sikkerhetsberegninger og simuleringer før oppskyting. Det er opp til hver enkelt hvorvidt disse gjøres før, under eller etter designfasen, men min anbefaling er å gjøre beregninger i starten av designfasen slik at man har grunnlag for å gjøre gode designvalg underveis, og til slutt gjør man avsluttende beregninger rett før oppskyting. Erfaringsmessig er det veldig lurt å ha beregninger knyttet til massesentrum, aerodynamikk, vekt og valg av motor på plass før oppskyting for å hjelpe elever med å gjøre gode designvalg. Dette kan vi gjøre ved hjelp av den gratis simuleringsprogramvaren OpenRocket.

Steg 1 for å gjøre simuleringer i OpenRocket er å gjenskape raketten digitalt. Dette gjøres ganske enkelt ved å velge hvilke deler raketten består av, endre geometrien slik at den er korrekt og justere massen og massesentrum for hver enkelt del. Vedlagt til denne sikkerhetsveiledningen er filen *Skisse_cansatrakett_5.ork* slik at du kan gjenskape raketten i figur 6 og bearbeide designet med utgangspunkt i den.



Figur 6 – Rakett designet i OpenRocket

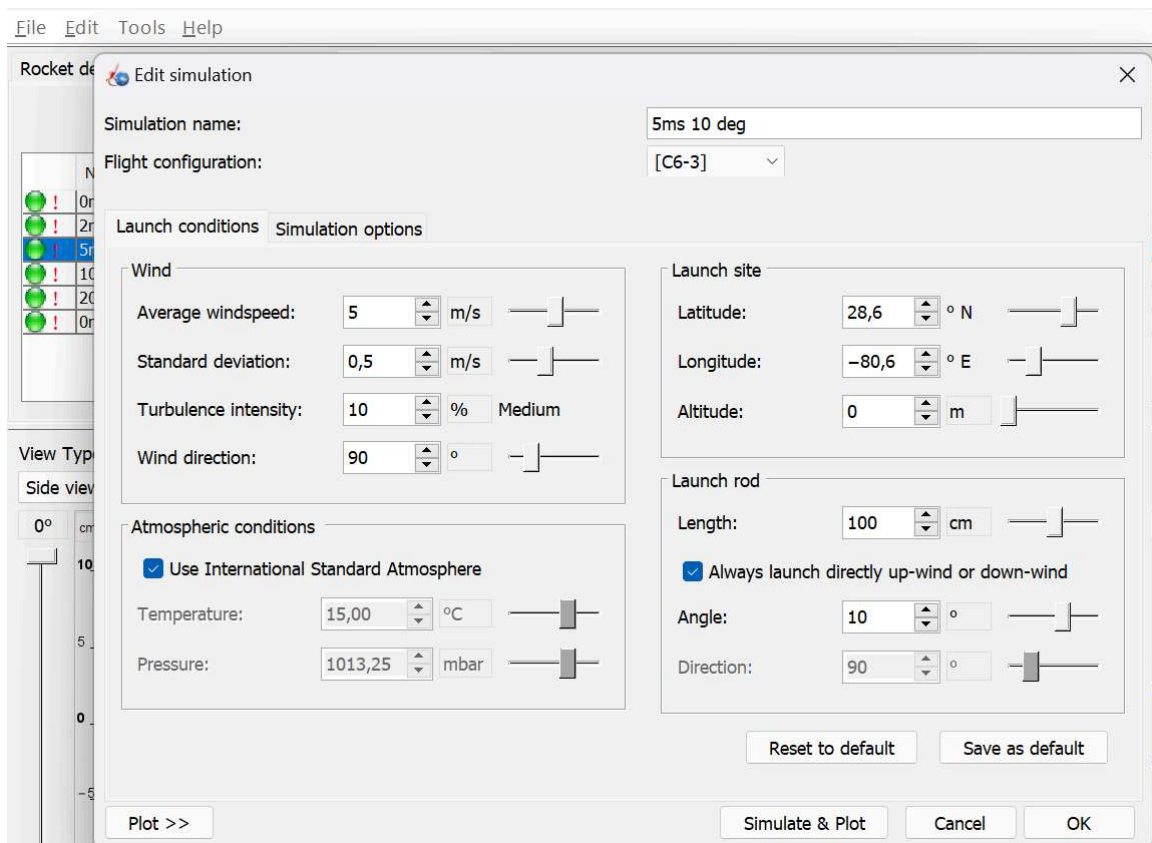
Når raketten er gjenskapt i Openrocket må det gjøres simuleringer. Dette kan man gjøre på fanen «Flight simulations». Her kan man simulere ulike oppskytingsforhold som vinkler, atmosfærisk trykk, turbulens, vindkast, vindretning og vindstyrke. Figur 7 viser at eksempel raketten vår vil gå 105 meter dersom den skytes rett opp i vindstille vær, mens hvis det er vindstille og man skyter med en vinkel på 10 grader vil raketten nå 101m. For å lettere kunne lage trygge sikkerhetssoner anbefales det å alltid ha en oppskytingsvinkel og ikke skyte raketter rett opp. Denne vinkelen bør også gå direkte inn i vinden. Raketten vil peke ytterligere inn i vinden under oppskyting, men vil trekkes tilbake med vinden når fallskjermen løses ut. Vi kan se at alle parametrene til raketten fremstår forsvarlige ved vind opp til 10ms, mens både apogee (maksimal høyde) og optimal forsinkelse på fallskjerm fremstår problematisk med en vindhastighet på 20ms. For å være ekstra forsiktig kan det i dette konkrete tilfellet være lurt å utsette oppskytingen hvis vinden er over 5 meter pr sekund.

Rocket design Motors & Configuration Flight simulations										
New simulation Edit simulation Run simulations Delete simulations Plot / Export										
Name	Configuration	Velocity off rod	Apogee	Velocity at deploym...	Optimum delay	Max. velocity	Max. acceleration	Time to apogee	Flight time	Ground hit velocity
0ms 10deg	[C6-3]	11,3 m/s	101 m	9 m/s	3,37 s	40,9 m/s	78,2 m/s ²	4,97 s	23,6 s	5,5 m/s
2ms 10deg	[C6-3]	11,3 m/s	95,2 m	11 m/s	3,22 s	40,9 m/s	78,1 m/s ²	4,87 s	22,6 s	5,5 m/s
5ms 10deg	[C6-3]	11,3 m/s	79,8 m	14,5 m/s	2,84 s	40 m/s	78 m/s ²	4,54 s	19,6 s	5,52 m/s
10ms 10deg	[C6-3]	11,3 m/s	65,6 m	15,9 m/s	2,43 s	38,5 m/s	78 m/s ²	4,13 s	16,7 s	5,6 m/s
20ms 10deg	[C6-3]	11,3 m/s	45,7 m	15,4 m/s	1,86 s	34 m/s	86,3 m/s ²	3,51 s	12 s	6,01 m/s
0ms 0deg	[C6-3]	11,2 m/s	105 m	4,47 m/s	3,47 s	40,7 m/s	78 m/s ²	5,03 s	24,4 s	5,3 m/s

Figur 7- Simuleringer av ulike oppskytingsbetingelser i OpenRocket

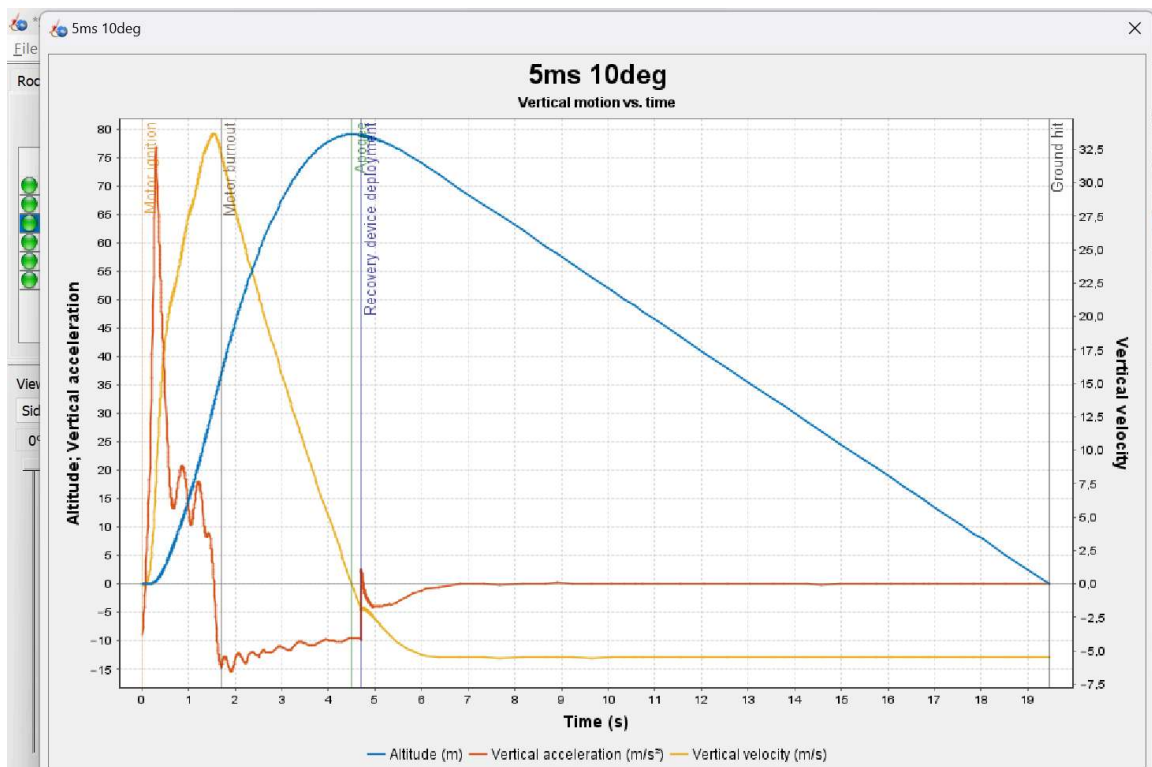
Nedenfor kan vi se ulike simuleringsparametre. Det er gjennomsnittlig Average windspeed og angle (oppskytingsvinkel) som vil være de mest relevante kriteriene. I skolesammenheng

vil jeg fraråde å skyte opp en modellrakett uten først å ha simulert oppførselen og forholdene på oppskytingsdagen i OpenRocket.



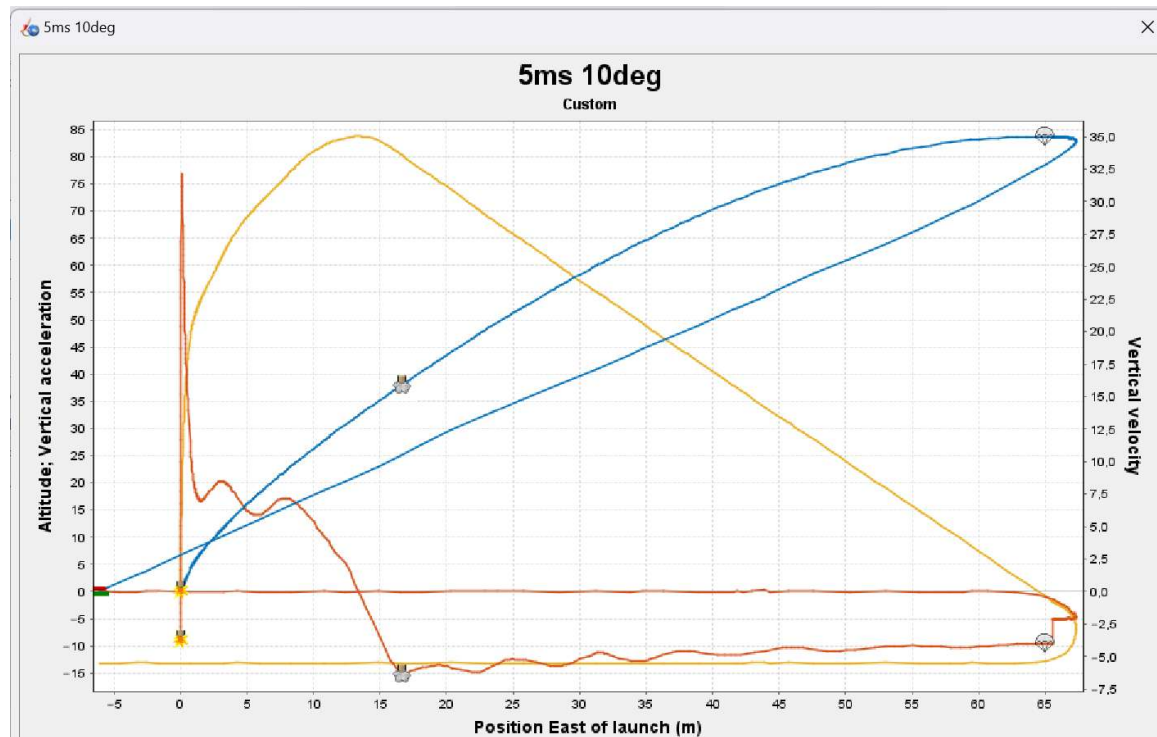
Figur 8 - Vindu som viser de vanligste simuleringsparametrene

Når vi har lastet inn alle parametre kan vi velge å simulere og plote oppskytingen. Dette gir oss anledning til å visualisere hele oppskytingsprosessen og se både høyder, akselerasjon og hastigheter.



Figur 9 - Plot fra simulering av oppskyting

Figur 10 viser posisjonen til raketten underveis i flyvningen. Dette er et svært nyttig plot for å si noe om hvor raketten vil lande. Som vi kan se av den blå grafen på plottet forventer vi at raketten er 60 meter øst fra launch pad når fallskjemen løses ut (ca 80 meters høyde), men den vil blåses tilbake til 6 meter vest for launchpad før den lander i denne simuleringen.



Figur 10 - Simulering av posisjon under oppskyting

Sjekkliste før simulering i OpenRocket (for å eliminere de vanligste feilene i input):

1. Alle komponenter har riktig geometri og masse
2. Korrekt valg av rakettmotor og korrekt forsinkelse for fallskjemutløsning
3. Korrekt beregnet fallskjemstørrelse og fallskjermgeometri
4. Korrekt form og størrelse på finner og andre aerodynamiske overflater
5. Korrekt barometrisk trykk, vindhastighet og oppskytingsvinkel

Sjekkliste for å tolke simulering i OpenRocket:

1. Kontroller forsvarlig høyde (mellom 80 og 120 meter vil ofte være passe for skoleprosjekter, men husk at å skyte raketten for lavt også kan være farlig)
2. Kontroller forsvarlig hastighet ved slutt på launch rod (ofte rundt 10m/s, saktere enn dette kan være problematisk)
3. Kontroller forsvarlig hastighet ved fallskjemutløsning (anbefalt under 20m/s)
4. Kontroller at den planlagte sikkerhetssonen er tilstrekkelig stor at raketten lander innenfor, selv dersom fallskjemen ikke utløses. Du kan enten simulere med en plugged-

versjon av rakettmotoren eller sette recovery device delay til lengre enn den totale flytiden. I mitt tilfelle var en sikkerhetssone i oppskytingsretningen på 200m tilstrekkelig.

Andre beregninger

Andre ledd av sikkerhetsberegningene kan brukes til å kvalitetssikre simuleringer fra OpenRocket. OpenRocket er svært avansert og pålitelig programvare, men vi kan ha vært uheldige under designet, eller ha aktivert funksjoner vi ikke har oversikt over. Derfor er det anbefalt å ha et ekstra sikkerhetsnett. Jeg anbefaler enten å utvikle selv, eller benytte seg av en numerisk baneberegning som hensyntar luftmotstand. Vedlagt er et strålende python-program utviklet av Fredrik B. Pettersen hos Andøya Space Education ved navn *NumeriskRakett_eksempel.py*. Det anbefales å bruke versjonen uten ferdigutfylte variabler (*NumeriskRakett_mal.py*) slik at du og elevene manuelt må fylle inn alle variabler, både for maksimalt læringsutbytte og for å sikre at det brukes korrekte variabler.

```
In [8]: # -*- coding: utf-8 -*-
        """
        Dette koden er rammeverket for en enkel numerisk beregning av rakettbaner. I denne
        er x retning horisonalt, mens y retning vertikalt. For at koden skal fungere må føl
        - konstanter for utskytingsparametre, rakettmotor, vekt og aerodynamikk må fyll
        - uttrykk for baneberegninger i while-løkkene må skrives inn. Her skal Newtons
        bevegelsesligningene for å finne hastighet og posisjon

        Andøya Space Education

        Opprettet: 11.10.2023
        Sist endret: 11.10.2023
        @forfatter: Fredrik B. Pettersen
        """

        # ===== #
        # ===== Blblioteker ===== #
        import numpy as np # for matematiske beregninger
        import matplotlib.pyplot as plt # for plotting

        # ===== #
        # ===== Konstanter ===== #
        # UTSKYTINGSPARAMETRE (SKRIV INN VERDIER!)
        theta = 80 # planlagt elevasjon (utskytningsvinkel) målt i grader
        g = 9.81 # standard tyngdeakselerasjon i m/s^2

        # RAKETTMOTOR (SKRIV INN VERDIER!)
        T_avg = 6.765 # gjennomsnittelig skyvekraft i Newton
        t_b = 1.565 # motorens brenntid i sekunder
        t_d = 3 # motorens glidetid i sekunder

        # VEKT (SKRIV INN VERDIER!)
        m_0 = 0.145 # raketts totale startmasse inkludert motor med drivstoff i kg
        m_p = 0.0095 # drivstoffmassen i kg
        m_avg = m_0 + m_p/2 # gjennomsnittelig masse i kg

        # AERODYNAMIKK (SKRIV INN VERDIER!)
        C_D = 0.81 # raketts luftmotstandskoeffisient
        rho = 1.225 # lufttetthet i kg/m^3
        A = np.pi*0.0185**2 + 4*(0.048*0.0015 + 0.002*0.0065) # raketts største tverrsnit
        C_D_P = 1.75 # fallskjermens luftmotstandskoeffisient
        A_P = np.pi*0.1**2 # fallskjermens areal i m^2
```

```

# print(A)

# SIMULERINGSKONSTANTER
dt = 0.01 # tidssteg mellom hver beregning målt i sekund
t_f = 100 # total simuleringstid mål i sekund

# ===== #
# ===== Initialisere numeriske beregninger ===== #

# Elevasjonen regnes om til radianer
theta = theta*np.pi/180

# Alle verdier som skal simuleres må ha en liste hvor hver enkelt beregning kan leg
t = np.arange(t_f, step = dt) # tid (starter på 0 s)
a_x = np.zeros(np.shape(t)) # horisontal akselerasjon (starter med 0 m/s^2)
a_y = np.zeros(np.shape(t)) # vertikal akselerasjon (starter med 0 m/s^2)
v_x = np.zeros(np.shape(t)) # horisontal hastighet (starter med 0 m/s)
v_y = np.zeros(np.shape(t)) # vertikal hastighet (starter med 0 m/s)
x = np.zeros(np.shape(t)) # Lengde (starter fra 0 m)
y = np.zeros(np.shape(t)) # høyde (starter fra 0 m)
m = np.full(np.shape(t), m_0) # masse (starter med startmasse)

# definerer en indeks (i) som holder styr på hvor vi er i listene våre
i = 0 # starter på 0
i_max = np.size(t)-1 # maksimal indeks i listene våre; hvis i blir større er vi ute

# konstantfaktorer i beregningene KAN defineres utenfor løkkene for å spare tid
beta = 0.5*C_D*rho*A # konstant i uttrykket for raketts luftmotstand
beta_P = 0.5*C_D_P*rho*A_P # konstant i uttrykket for fallskjermens luftmotstand
T_x = T_avg*np.cos(theta) # konstant skyvekraft fra motoren i x-retning
T_y = T_avg*np.sin(theta) # konstant skyvekraft fra motoren i y-retning

# ===== #
# ===== Numeriske beregninger ===== #

while t[i] < t_b: # Baneberegninger mens motoren brenner, dvs. når t er mindre enn
    # Fyll inn uttrykk for akselerasjon i x- og y-retning (Newtons 2 Lov)
    a_x[i+1] = 1/m_avg*(T_x - beta*v_x[i]*np.sqrt(v_x[i]**2 + v_y[i]**2))
    a_y[i+1] = 1/m_avg*(T_y - beta*v_y[i]*np.sqrt(v_x[i]**2 + v_y[i]**2)) - g

    # Hastighet i x- og y-retning. Bruk bevegelsesligningen for hastighet
    v_x[i+1] = a_x[i]*dt + v_x[i]
    v_y[i+1] = a_y[i]*dt + v_y[i]

    # Posisjon i x- og y retning. Bruk bevegelsesligningen for posisjon
    x[i+1] = 0.5*a_x[i]*dt**2 + v_x[i]*dt + x[i]
    y[i+1] = 0.5*a_y[i]*dt**2 + v_y[i]*dt + y[i]

    # Oppdater i. Hvor hver beregning øker indeksen med 1
    i += 1

while t[i] <= t_b + t_d: # Baneberegninger når raketten er i glideflukt. Dvs. motor
    # Akselerasjon
    a_x[i+1] = 1/m_avg*(- beta*v_x[i]*np.sqrt(v_x[i]**2 + v_y[i]**2))
    a_y[i+1] = 1/m_avg*(- beta*v_y[i]*np.sqrt(v_x[i]**2 + v_y[i]**2)) - g

    # Hastighet
    v_x[i+1] = a_x[i]*dt + v_x[i]
    v_y[i+1] = a_y[i]*dt + v_y[i]

    # Posisjon
    x[i+1] = 0.5*a_x[i]*dt**2 + v_x[i]*dt + x[i]

```

```

y[i+1] = 0.5*a_y[i]*dt**2 + v_y[i]*dt + y[i]

# Oppdater i
i += 1

while i < i_max and y[i-1] >= 0: # etter at fallskjermen har kommet ut. Dvs. når ti
    # Aksekerasjon
    a_x[i+1] = 1/m_avg*(- beta_P*v_x[i]*np.sqrt(v_x[i]**2 + v_y[i]**2))
    a_y[i+1] = 1/m_avg*(- beta_P*v_y[i]*np.sqrt(v_x[i]**2 + v_y[i]**2)) - g

    # Hastighet
    v_x[i+1] = a_x[i]*dt + v_x[i]
    v_y[i+1] = a_y[i]*dt + v_y[i]

    # Posisjon
    x[i+1] = 0.5*a_x[i]*dt**2 + v_x[i]*dt + x[i]
    y[i+1] = 0.5*a_y[i]*dt**2 + v_y[i]*dt + y[i]

    # Oppdater i
    i += 1

# Raketten har krasjet, så vi sletter unødvendige data
t = t[:i]
x = x[:i]
y = y[:i]
v_x = v_x[:i]
v_y = v_y[:i]
a_x = a_x[:i]
a_y = a_y[:i]
m = m[:i]

# ===== #
# ===== Plotting ===== #

# Først lukker vi alle åpne figurer, slik at vi slipper å gjøre det manuelt
plt.close('all')

# Høyde over Lengde
plt.figure('Høyde over avstand')
plt.plot(x,y)
plt.xlabel("Lengde [m]")
plt.ylabel("Høyde [m]")
plt.gca().set_aspect(1) # Sørger for at forholdet mellom aksene er 1:1.
plt.grid(linestyle='--')
plt.show()

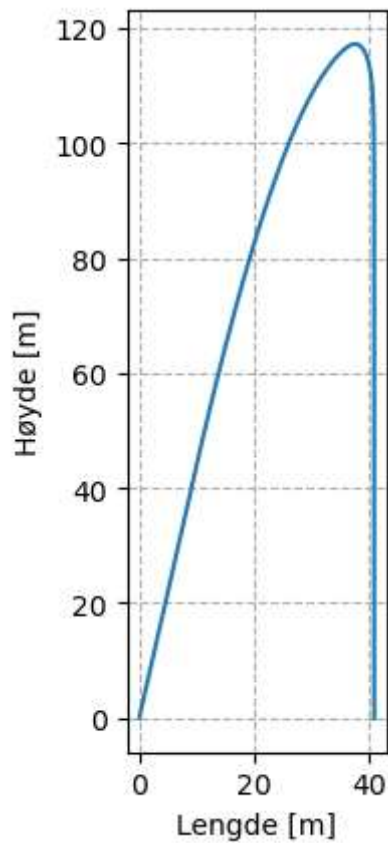
# Høyde over tid
plt.figure('Høyde over tid')
plt.plot(t, y)
plt.xlabel("Tid [s]")
plt.ylabel("Høyde [m]")
plt.grid(linestyle='--')
plt.show()

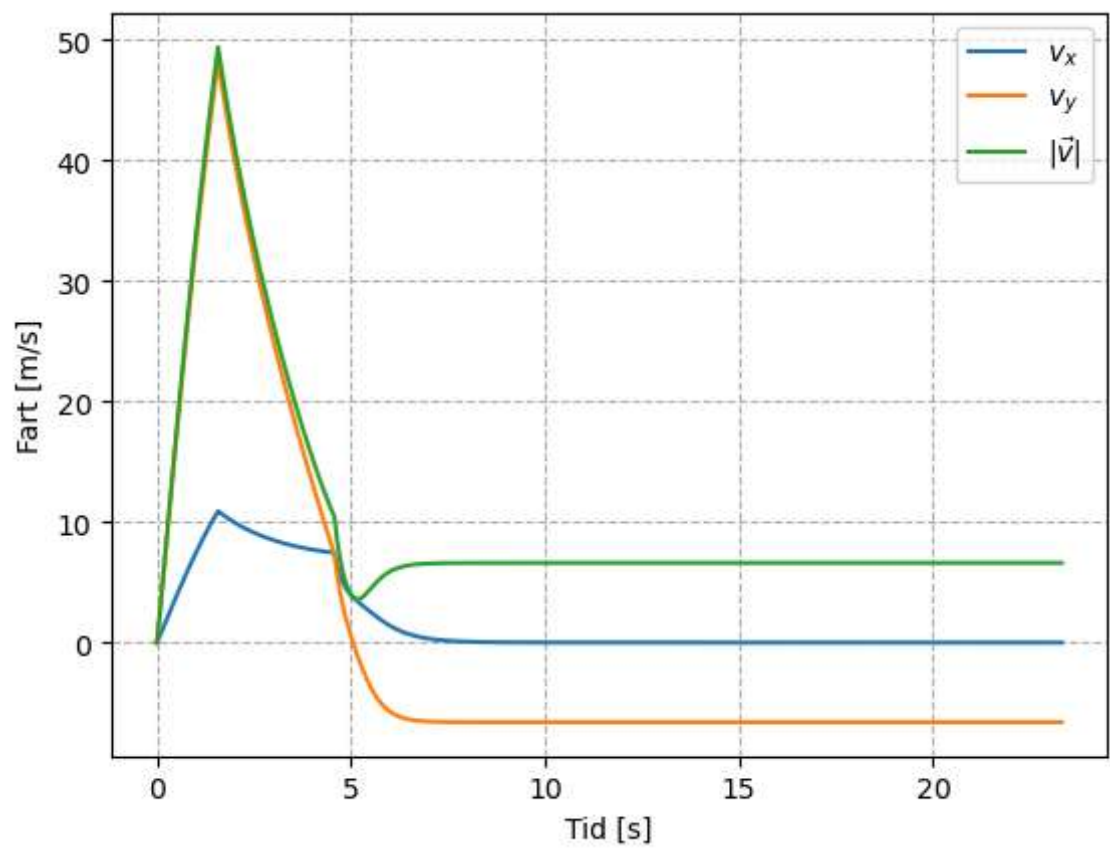
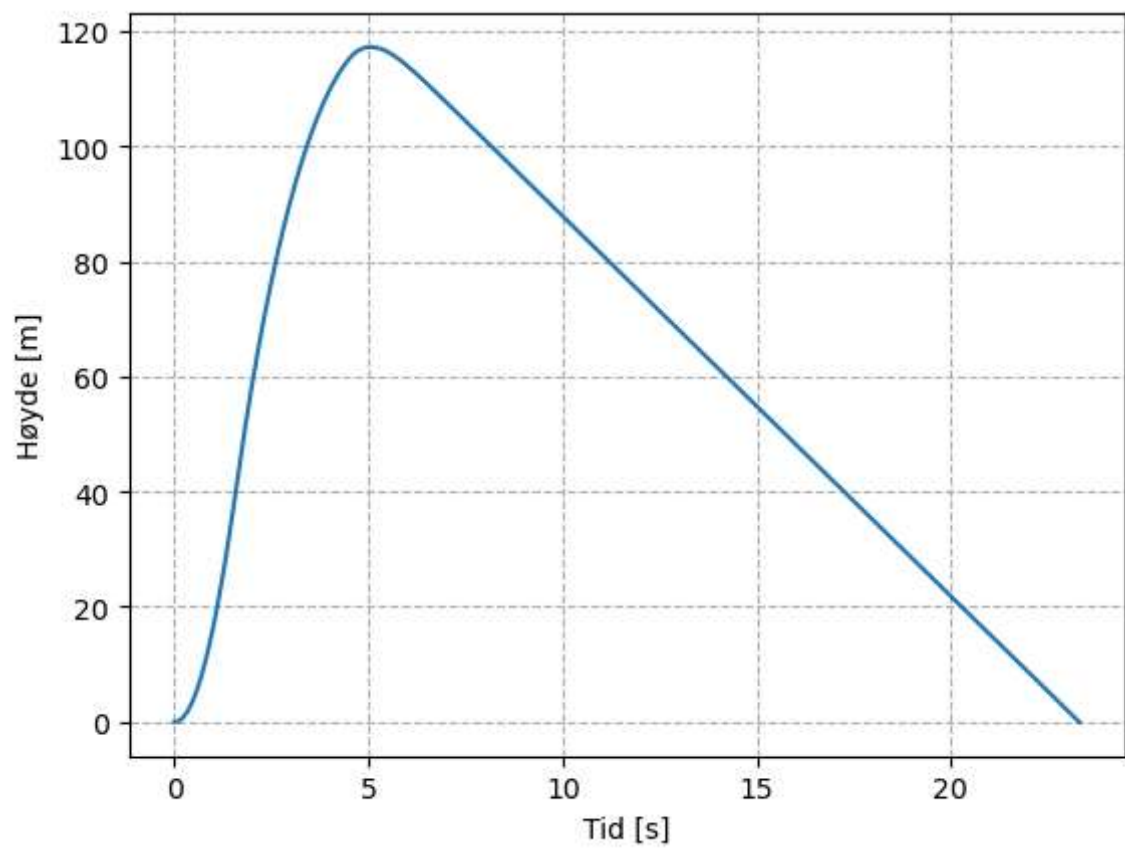
# Hastighet over tid
plt.figure('Hastighet over tid')
plt.plot(t, v_x, label='$v_x$')
plt.plot(t, v_y, label='$v_y$')
plt.plot(t, np.sqrt(v_x**2 + v_y**2), label=r'$|\vec{v}|$')
plt.xlabel("Tid [s]")
plt.ylabel("Fart [m/s]")
plt.grid(linestyle='--')
plt.legend()
plt.show()

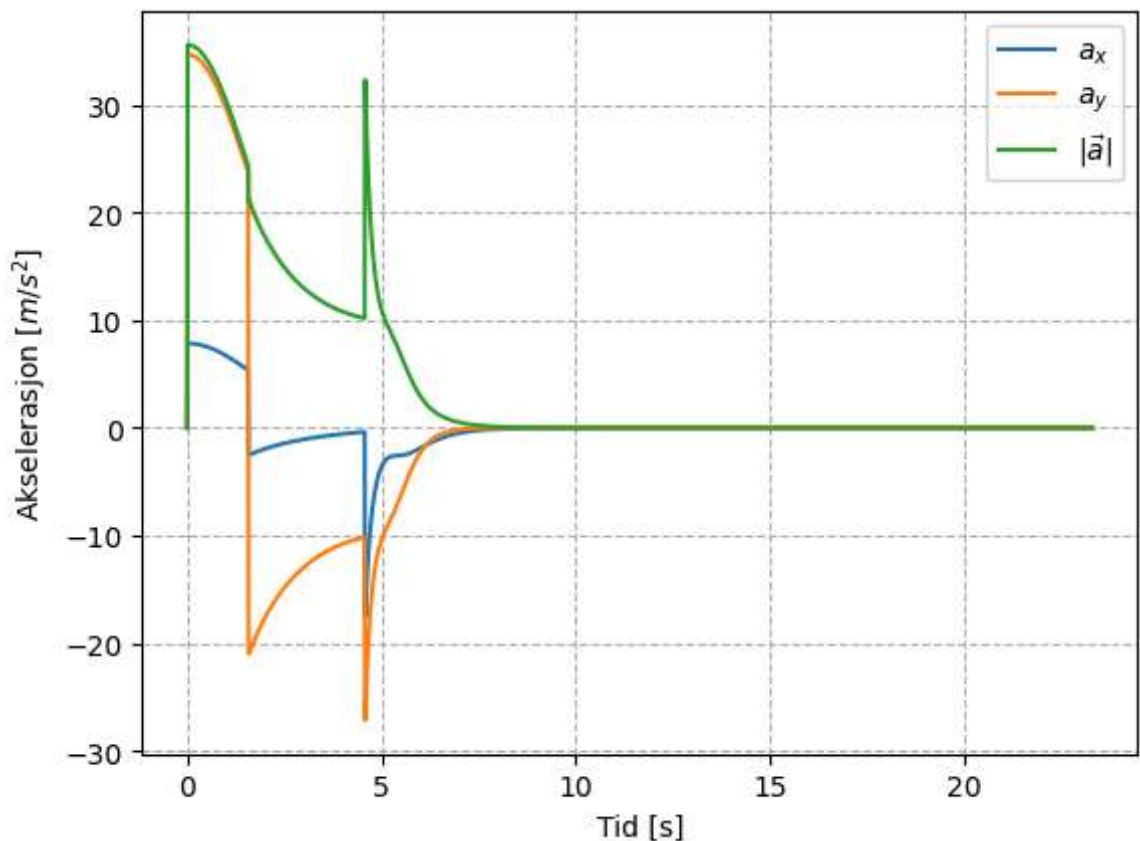
```



```
# Akselerasjon over tid
plt.figure('Akselerasjon over tid')
plt.plot(t, a_x, label='$a_x$')
plt.plot(t, a_y, label='$a_y$')
plt.plot(t, np.sqrt(a_x**2 + a_y**2), label=r'$|\vec{a}|$')
plt.xlabel("Tid [s]")
plt.ylabel("Akselerasjon [m/s^2]")
plt.grid(linestyle='--')
plt.legend()
plt.show()
```







Figur 11 - Fredrik B. Pettersens NumeriskRakett_eksempel.py

Denne simuleringen brukes til å verifisere beregningene fra OpenRocket, så jeg ville kun gått videre med raketten når vi har sikret at simuleringen fra python-skriptet beskriver raketter med relativt lik oppførsel. Vi forventer altså stor grad av sammenfall mellom de tre simuleringene våre:

Forenklet simulering uten luftmotstand

OpenRocket-simuleringer

Baneberegningsskriptet

Dersom alle disse beskriver en rakett med trygg oppførsel ihht krav kan raketten ferdigstilles for ytterligere fysisk testing.

Beregninger før oppskyting

Husk å gjøre nye beregninger før oppskyting. Dersom man har med laptop med OpenRocket, eller bruker remote desktop-programvare på telefonen kan man enkelt korrigere for de faktiske forholdene under oppskytingen. Husk at vindretningen kan være annerledes ved 100m enn på bakken. Ved usikkerhet rundt vindretning bør ikke raketten skytes opp.

Manuelle sikkerhetsberegninger

Jeg skal ikke gå i detalj på alle fysiske sikkerhetsberegninger her, men etter raketten er bygget må alle fysiske deler inspiseres for feil og uregelmessigheter. I tillegg bør det gjøres

manuell beregning av center of mass (raketten balanseres på en finger for i finne vippepunktet) og sjekke at dette ligger minst et kaliber (en diameter av raketten) foran center of pressure.

Center of pressure kan beregnes ved en såkalt skyggemodell. [1] Da tegnes skyggen til raketten på et ark og du finner det geometriske gjennomsnittet av skyggefiguren. Hvis man kutter ut skyggen i papp eller plast kan man også finne vippepunktet til skyggefiguren.

Jeg vil også anbefale å sjekke stabilitet ved snor-fremgangsmåten[2]. Da fester man en snor rundt raketten og snurrer den rundt seg for å se om den flyr stabilt. Begynn ved center of mass og flytt snoren gradvis bakover til du får ustabil oppførsel fra raketten for å se hvor god margin du har. Til slutt må det gjøres en avsluttende inspeksjon. Dette inkluderer alle deler av oppskytingssystemet, raketten payload, fallskjerm, rakettmotor, oppskytingsrampe, sikkerhetsutstyr, avfyringspanel.

Andre sikkerhetstiltak og sjekkliste

Før oppskytingsdagen

Innhent tillatelse fra grunneier til rakettoppskyting.

Meld arrangement til politiet digitalt i god tid før oppskytingen (gjørne mer enn 14 dager)

Dokumenter sikkerhetsrutiner og sikre godkjennelse fra rektor for oppskyting. Dette sikrer at du ikke står alene om å gå god for sikkerhetsberegningene

Gjennomgå sikkerhetsprotokoll med lærere i forkant

Det anbefales å ta spesielt stort hensyn i perioder med skogbrannfare. Det anbefales å ikke skyte opp raketter i bålforbudssesongen uten å ha avklart dette med lokalt brannvesen.

På oppskytingsdagen

Varsle politiet på 02800 ca 30 minutter før oppskyting

Gjennomgå alle sikkerhetsprotokoller med alle deltakere, inkludert rutiner ved hangfire og avvik fra forventet bane.

Etabler sikkerhetssone med vakter slik at uvedkommende ikke kan komme inn i området uten å bli varslet

Sørg for at nødvendig førstehjelpsutstyr, sikkerhetsutstyr og brannslukningsutstyr er tilgjengelig

Ekstra sikkerhetstiltak - Avfyringspanel

For å kunne gjennomføre oppskyting av modellraketter i skolen på en trygg måte er det nødvendig med et avfyringssystem. Dette sikrer at alle involverte i oppskytingen kan holde tilstrekkelig sikkerhetsavstand. Avfyringssystemet består av en nedtelling fra 5 som går så lenge begge knappene på en micro:bit holdes inne. Dersom en (eller begge) knappene slippes under nedtellingen nullstilles nedtellingen. Dersom nedtellingen fullføres sendes et signal til et relé som antenner en elektronisk lunte for modellraketten.

Tips til lærer:

Dersom elevene er erfarne med micro:bit eller dere har god tid anbefales det å la elevene eksperimentere med å lage programmet for mottaker og sender selv. Elevene kan lage flytskjema og programmere senderen individuelt, i grupper eller i plenum. Mengden veiledning som er nødvendig vil avhenge av elevgruppen. I den første klassen der jeg prøvde ut denne programmeringsøkten hadde jeg god erfaring med å demonstrere det ferdige produktet, men la elevene selv utforske programmeringen med individuell veiledning underveis ved behov.

Skjema for radiogruppe og avfiringssignal. Fylles ut i forkant av programmeringsøkt.

Det er anbefalt å la radiogruppe matche gruppenummer for å forenkle tildeling av radiogrupper.

Gruppe-nr	Rakettnavn	Radiogruppe	Avfiringssignal

Utstyr:

Avfyringspanel (bygges av lærer eller elever i forkant av gruppeøvelsen)

1x OCTOPUS 1 Channel 3V Relay Module

1x breakout board

1x 9v-batteri

1x batteripakke til micro:bit

2x klemmer for tilkobling til elektrisk lunte

Egnede ledninger

Utstyr pr gruppe

2x micro:bit

1x usb-kabel til micro:bit

Merkelapper for tydelig merking av micro:bit med radiogruppe og «mottaker» og «sender».

PC eller iPad med tilgang til micro:bit makecode

Programmering av sender:

Her eksempelkode for programmering av sender:

Programmering av mottaker

Bygging av avfyriingspanel

Det finnes mange måter å bygge et avfyriingspanel for modellraketter. Jeg har valgt å bruke et ElecFreaks OCTOPUS 1 Channel 3V Relay Module koblet til et 9V-batteri. Her vil et signal fra micro:bit koble inn 9V-batteriet slik at det motoren til raketten.

Testing av avfyriingspanel:

Viktig sikkerhetsinformasjon: Ettersom mottakeren skal brukes til å fyre av en potensielt farlig sprengladning må hver enkelt sender og mottakers kode kontrolleres og testes før bruk. Her er en sjekkliste.

Lærer har ansvar for å gjennomføre følgende i forkant av bruk av avfyriingspanelet:

- Inspiser overføring av nyeste versjon av elevenes kode til sender og mottaker.
- Inspiser at koden ser hensiktsmessig ut. Dersom lærer ikke kan gå god for at koden gjennomfører oppgaven den skal på en trygg måte må den revideres før ny overføring.
- Nedtellingstid er minimum 2.5 sekunder ($5 \cdot 500ms$) med visuell nedtelling på sender og mottaker.
- Kontroller at det er valgt riktig utgang for avfyriingspanel (samsvar mellom kode og kobling av avfyriingspanel)
- Kontroller at avfyriingssekvens avhenger av en korrekt overført tekststreng (for å unngå falske positive signaler).
- Tester: Bruk elektromotor, lyspære eller annet egnet utstyr for å teste om avfyriingspanelet faktisk aktiveres.
 - Test av avbrutt nedtelling
 - Test av omstart under nedtelling
 - Test av gjennomført nedtelling.
- Nedtellingsprotokoll gjennomgås med elever slik at de er kjent med rutinene for oppskyting og sikker bruk.

Evaluerings av sikkerhetstiltak og kalibrering av beregninger etter oppskyting

Etter gjennomført oppskyting er det viktig å både evaluere sikkerhetstiltakene og måle hvor presise beregningene var. Programmer for å analysere innsamlede data fra f.eks en brusbokssatelitt eller annen datalogger. Her er et eksempel fra min siste oppskyting med en rakett bygget med en datalogger som registrerte trykk, temperatur, luftfuktighet, akselerasjon, magnetfelt etc. Beregninger med både OpenRocket og python-programmet for numerisk beregning av oppskytingshøyde viste forventet toppunkt på 111-117 meter.

Reell høyde var 112-114 meter ifølge dataloggeren om bord og en separat høydemåler. Både Python-programmet og OpenRocket var altså tilstrekkelig presise hver for seg, men med kun én simuleringsplattform til å gjøre beregningene ville jeg ikke følt meg trygg nok på simuleringene til å gjennomføre oppskytingen ettersom veldig små detaljer kan få store utslag.

Figur - Analyse av trykk/høydedata fra modellrakettoppskyting

Lykke til!

Forhåpentligvis føler du deg klar for å gjøre sikkerhetsberegninger som skal til for å bruke modellraketter i skolen! Modellraketter lar elever lære om fysikk i praksis, og med grundig forarbeid er det en trygg og engasjerende måte å få elevene til å jobbe tverrfaglig innen realfagene!

Vedlegg

Vedlegg 1: Nedtellingsprosedyre

Vedlagt er forslag til nedtellingsprosedyre. Denne er hentet fra Andøya Space Education med tillatelse. Avhengig av alder og modenhetsnivå hos elevene kan det være nødvendig at lærer innehar alle rollene beskrevet nedenfor under selve oppskytingen, selv om det mest engasjerende for elevene naturligvis er om de får være delaktige selv.

Ansvar og roller under oppskyting

Rolle	Forkortelse	Hvem?	Ansvar
Range safety officer	RSO		- sørger for at sjekklista i nedtellingsprosedyren blir fulgt, og krysser etterhvert som oppgaver blir utført - ansvarlig for å se hvor raketten lander. Krever at man holder øye med raketten gjennom hele flukten!
Launch control	LC		- ansvarlig for å sette raketten på rampa, koble tenningsystem og montere lunte - gjennomføre nedtelling høyt og tydelig fra 5 - trykke på avfyringsknappen
Payload manager	PM		- sørge for at nyttelasten er koblet til strøm, skriver data og er sikkert plassert i raketten - ansvarlig for eventuell videodokumentasjon av rakettskuddet.
Instructor/teacher	INS		tar med rampe, tenner, lunter og vernebriller til oppskytingssted

Rolle	Forkortelse	Hvem?	Ansvar
			godkjenner rakett i henhold til sjekkliste i nedtellingsprosedyre

Nedtellingsprosedyre

Tidspunkt relativt til oppskytning HH-MM-SS	Nr. #	Hvem utfører og hvem rapporteres det til		Kryss når utført	Ingen kritisk status	Beskrivelse av oppgave
					Nyttelast på intern strøm	
					Rakett på rampen	
					Tenning armert	
Fra	Til					
00-45-00	1	RSO	ALL			Sjekk værmeldingen for oppskytingssted ved planlagt tidspunkt for oppskyting
	2	RSO	ALL			Bekreft at simuleringer og baneberegninger for raketten er gjennomført og godkjent
	3	ALL	RSO			Les vedlegget «Sikker oppskyting av modellraketter» og gjør deg godt kjent med oppgavene i denne nedtellingsprosedyren
	4	INS	ALL			Raketten skal godkjennes av instruktør. Instruktør sjekker at: - o raketten er bygget i henhold til instruksjonene - o riktig motor montert - o nyttelast og rakettkropp kan separeres enkelt - o finnene er festet tilstrekkelig - o motorfestet er limt godt fast - o knutene mellom rakett og fallskjerm ser bra ut - o raketten inneholder wadding eller annen flamedemper for beskyttelse av fallskjerm
00-02-00	5	ALL	RSO			Ta på vernebriller
	6	RSO	ALL			Sjekk at alle tilskuere er på trygg avstand fra rakettrampen, minimum 15 meter
	7	RSO	ALL			Sjekk at værforholdene er innenfor kriteriene for oppskyting
	8	LC	RSO			Sett raketten på rampen
	9	PM	RSO			Skru på nyttelasten og bekreft at lysindikator er aktivert
	10	LC	RSO			Koble tenningsssystemet til lunta
	11	LC	RSO			Sett lunta inn i rakettmotoren
00-00-30	12	RSO	ALL			Sjekk at alle på gruppa er på trygg avstand fra rakettrampen, minimum 5 meter
	13	PM	RSO			Bekreft at PM er klar for videodokumentasjon av rakettskuddet
	14	LC	RSO			Armer tenningsssystemet
	15	LC	ALL			Tell ned fra T -5 høyt og tydelig
00-00-00	16	LC				Oppskyting
	17	RSO				Visuell sporing av raketten helt til den har landet
	18	RSO	ALL			Bekreft landing og husk landingssted for å gjøre henting/leting enklere
	19	LC	RSO			Desarmer tenningsssystemet
	20	ALL	RSO			Legg vernebriller på plass
	21	ALL				Hent raketten når alle rakettenes er skutt opp
	22	PM	RSO			Skru av nyttelast når raketten er funnet

Sikker oppskyting av modellraketter

Denne sjekklisten er utviklet av Andøya Space Education med utgangspunkt i Luftfartstilsynets forskrift om luftfartøy som ikke har fører ombord og The National Association of Rocketry (NAR) sine retningslinjer for sikker oppskyting av modellraketter. Denne veiledningen er laget for elever, lærere, vitensenterpedagoger og andre modellrakettentusiaster som en retningslinje for trygg bruk av modellraketter.

Generelle bestemmelser fra forskrift om luftfartøy som ikke har fører om bord

Modellraketter faller under definisjonen modellfly som vi finner under §4 Definisjoner og forkortelser i forskrift om luftfartøy som ikke har fører om bord. Derfor forholder vi oss til de generelle bestemmelsene i denne forskriften som gjelder modellfly.

«Modellfly: innretninger som ikke har fører om bord, som beveger seg i luften og hvor flygingen kun har rekreasjon, sport eller konkurranse som formål» – §4 Definisjoner og forkortelser, Forskrift om luftfartøy som ikke har fører om bord.

Ifølge §6 Begrensninger for flygningen skal all flyging med modellfly skje på en hensynsfull måte som ikke utsetter luftfartøy, personer, fugler, dyr eller eiendom for risiko for skade eller for øvrig er til sjenanse for allmennheten.

Modellflyet må til enhver tid være godt synlig for den som fører det slik at full kontroll over modellflyet kan opprettholdes. Flyging med modellfly kan bare skje i dagslysperioden og ikke:

1. høyere enn 120 meter over bakken eller vannet
2. nærmere enn 150 meter fra personer, motorkjøretøy eller bygning som ikke er under fartøyførerens kontroll med unntak av under start og landing
3. nærmere enn 5 km fra lufthavner

Sjekkliste for oppskyting av modellraketter

1. Materialer: Modellrakettenes nesecone, kropp og finner skal bygges i lette materialer. Ikke metall eller andre tyngre materialer.
2. Motorer: Det skal kun brukes sertifiserte motorer fra seriøse leverandører. Les alltid medfølgende bruksanvisning nøye før bruk av rakettmotorer.
3. Antenning: Modellrakettenes skal alltid skytes opp med en elektrisk tenner og elektrisk lunte. Tennen skal ha en sikringsfunksjon som kan slås av og på. Avfyringsknappen skal av seg selv gå tilbake i av posisjon etter at den er trykket inn.
4. Feiltenning: Hvis raketten ikke antennes skal sikring til tennen slås på, eller batteriet til tennen fjernes. Deretter skal en vente i minst 240 sekunder før man nærmer seg raketten med nødvendig verneutstyr.
5. Sikkerhet ved oppskyting: Enhver oppskyting skal ha egen nedtelling slik at alle tilskuere er oppmerksomme på oppskytingen. Alle tilskuere skal holde minst 15 m avstand. Den som trykker på tennen skal bruke vernebriller, og ha en avstand på minst 5 m fra utskytingsplattformen.
6. Utskytingsrampe: Det skal brukes en dedikert rakettrampe, f.eks. en utskytningsplattform med startstang. Vinkelen til rakettrampen skal kunne justeres, og ved oppskyting skal vinkelen alltid være mellom 70° og 85° fra horisontalplanet, slik at raketten flyr omtrent rett opp. Det skal også brukes en varmedeflektor for å forhindre den varme gassen fra motoren i å treffe bakken. Ved bruk av startstang bør enden på stanga sikres mellom hver oppskyting for å forhindre øyeskader.

7. Størrelse: Rakettens startvekt skal ikke overgå 250 gram. Største totalimpuls på motoren skal ikke overgå 20 Ns (motorkategori D).
8. Sikkerhet under flygning: Modellrakter skal kun skytes i klar sikt. Raketter skal ikke skytes ved lavt skydekke eller i nærheten av fugler, fly eller andre objekter i lufta. Ved bruk av nyttelast skal ikke denne være brennbar, eksplosiv eller utgjøre noen annen fare for raketten eller omgivelsene.
9. Egnet sted for oppskyting: Raketter skal kun skytes utendørs i et stort åpent område med minst 200×200 m klaring, og hvor alle personer som oppholder seg innenfor dette området er informert om aktiviteten. Sjekk alltid værmeldingen før planlagt oppskyting av modellrakter. Rakettoppskytinger skal ikke skje ved middelvind som overgår 7 m/s. Oppskyting skal heller ikke skje i områder som er eksponert for skogbrann.
10. Fallskjerm: Enhver modellrakter skal ha fallskjerm slik at den kan lande sikkert. Det skal også alltid brukes et flammebestandig materiale inni raketten for å beskytte fallskjermens funksjon når denne skytes ut, f.eks. wadding.
11. Sikkerhet ved henting: Raketter skal ikke hentes hvis de lander på utilgjengelige steder, f.eks. høyt oppe i trær eller i strømledninger. Avhengig av situasjon skal relevant fagpersonell (strømselskap, politi, brannvesen e.l.) kontaktes

Vedlegg 2: Modellrakter – Aktiviteter for fysikken i fokus Andøya Space Education

https://github.com/MortenOsterud/modellrakter_i_skolen/blob/main/Modellrakter%20Aktivi

Vedlegg 3: Excel-fil for ROS-analyse

https://github.com/MortenOsterud/modellrakter_i_skolen/blob/main/Excel-fil%20for%20ROS-analyse.xlsx

Fotnoter

[1] [How to Find the Center of Pressure on a Rocket](#)

[2] [\(332\) Rocket Stability - YouTube](#)